

Лабораторная работа № 7.

CORS и безопасность HTTP-взаимодействия

Тема: Same-Origin Policy, CORS, preflight OPTIONS и credentialed requests

Цель: Настроить безопасное cross-origin взаимодействие frontend и Express API, исследовать CORS headers и preflight, ограничить разрешённые origins и выполнить учебный эксперимент с credentials и HttpOnly cookie.

Дисциплина: Node.js и Express.js

Исходные условия

Продолжайте проект express-shop. Backend запускается на `http://localhost:3000`, а frontend в этой работе должен открываться с `http://localhost:5500`. Эти адреса имеют разные origins, потому что отличаются ports.

Что проверяет CORS

CORS применяется браузером и управляет доступом JavaScript к cross-origin response. Он не заменяет JWT, `requireAuth`, `requireRole`, `validation` и другие server-side проверки.

1. Запустите frontend на отдельном port

```
PowerShell
npx serve frontend -l 5500
```

Откройте frontend по адресу `http://localhost:5500`. Backend оставьте на `http://localhost:3000`.

2. Сравните origins

Приложение	URL	Scheme	Host	Port
Frontend	<code>http://localhost:5500</code>	http	localhost	5500
Backend	<code>http://localhost:3000</code>	http	localhost	3000

Origins различаются из-за port, поэтому browser применяет Same-Origin Policy и CORS.

3. Временно отключите cors middleware

Если в `app.js` уже подключён пакет `cors`, временно закомментируйте его вызов и перезапустите backend.

```
app.js
// app.use(cors(...));
```

4. Выполните cross-origin GET

```
frontend/app.js

async function testProducts() {
  const response = await fetch(
    "http://localhost:3000/api/products"
  );

  const data = await response.json();
  console.log(data);
}

testProducts();
```

1. Откройте DevTools → Console.
2. Обновите frontend.
3. Зафиксируйте CORS error.
4. Откройте DevTools → Network.
5. Найдите запрос к `/api/products`.
6. Обратите внимание: CORS error возникает в browser-клиенте.

5. Установите cors

```
PowerShell

npm install cors
```

6. Разрешите только frontend origin

```
app.js

import cors from "cors";

app.use(cors({
  origin: "http://localhost:5500",
  methods: ["GET", "POST", "PATCH", "DELETE"],
  allowedHeaders: ["Content-Type", "Authorization"]
}));
```

Перезапустите backend и повторите GET. Browser должен разрешить frontend прочитать JSON response.

7. Проверьте CORS response header

7. Откройте запрос `/api/products` в DevTools → Network.
8. Перейдите в Headers.
9. Найдите `Origin` в Request Headers.
10. Найдите `Access-Control-Allow-Origin` в Response Headers.
11. Убедитесь, что разрешён именно `http://localhost:5500`.

8. Создайте cross-origin POST с JSON

```
frontend/app.js
```

```

async function testCreateProduct(accessToken) {
  const response = await fetch(
    "http://localhost:3000/api/products",
    {
      method: "POST",
      headers: {
        "Content-Type": "application/json",
        "Authorization": `Bearer ${accessToken}`
      },
      body: JSON.stringify({
        name: "USB Hub",
        price: 12000,
        categoryId: 1,
        inStock: true
      })
    }
  );

  console.log(response.status);
  console.log(await response.json());
}

```

Используйте JWT пользователя с ролью `admin`, потому что Product POST уже защищён authorization middleware.

9. Найдите OPTIONS preflight

12. Откройте DevTools → Network.
13. Запустите `testCreateProduct()`.
14. Найдите запрос с method OPTIONS.
15. Откройте его Request Headers.
16. Найдите Origin.
17. Найдите Access-Control-Request-Method.
18. Найдите Access-Control-Request-Headers.
19. Откройте Response Headers и найдите Access-Control-Allow-*

10. Зафиксируйте структуру preflight

Схематичный preflight request

```

OPTIONS /api/products HTTP/1.1
Origin: http://localhost:5500
Access-Control-Request-Method: POST
Access-Control-Request-Headers: authorization, content-type

```

Схематичный preflight response

```

HTTP/1.1 204 No Content
Access-Control-Allow-Origin: http://localhost:5500
Access-Control-Allow-Methods: GET,POST,PATCH,DELETE
Access-Control-Allow-Headers: Content-Type,Authorization

```

11. Проверьте запрещённый origin

Запустите второй frontend на другом port, например `http://localhost:5600`, и повторите запрос к API.

```
Второй PowerShell
```

```
npx serve frontend -l 5600
```

Backend разрешает только `http://localhost:5500`, поэтому JavaScript с `http://localhost:5600` не должен получить cross-origin доступ к response.

12. Вынесите разрешённый origin в .env

```
.env
```

```
FRONTEND_ORIGIN=http://localhost:5500
```

```
app.js
```

```
app.use(cors({
  origin: process.env.FRONTEND_ORIGIN,
  methods: ["GET", "POST", "PATCH", "DELETE"],
  allowedHeaders: ["Content-Type", "Authorization"]
}));
```

13. Создайте учебный credentialed endpoint

Следующий endpoint нужен только для исследования cookie/CORS и не заменяет текущую JWT-аутентификацию проекта.

```
app.js
```

```
app.post("/api/demo-cookie", (req, res) => {
  res.cookie("demo_session", "demo-value", {
    httpOnly: true,
    sameSite: "lax",
    maxAge: 10 * 60 * 1000
  });

  res.json({ message: "Demo cookie created" });
});
```

14. Включите credentialed CORS

```
app.js
```

```
app.use(cors({
  origin: process.env.FRONTEND_ORIGIN,
  credentials: true,
  methods: ["GET", "POST", "PATCH", "DELETE"],
  allowedHeaders: ["Content-Type", "Authorization"]
}));
```

Wildcard запрещён для credentialed CORS

При `credentials: true` разрешайте конкретный origin. Не используйте `Access-Control-Allow-Origin: *` для credentialed requests.

15. Отправьте fetch с credentials

```
frontend/app.js

async function createDemoCookie() {
  const response = await fetch(
    "http://localhost:3000/api/demo-cookie",
    {
      method: "POST",
      credentials: "include"
    }
  );

  console.log(await response.json());
}

createDemoCookie();
```

20. Откройте DevTools → Network.
21. Выполните запрос.
22. Проверьте `Access-Control-Allow-Origin`.
23. Проверьте `Access-Control-Allow-Credentials: true`.
24. Откройте DevTools → Application → Cookies.
25. Найдите `demo_session`.
26. Убедитесь, что cookie имеет `HttpOnly`.

16. Проверьте доступность HttpOnly cookie из JavaScript

```
Browser Console

console.log(document.cookie);
```

Значение `demo_session` не должно быть доступно через `document.cookie`, потому что cookie установлена с `HttpOnly`.

17. Сравните Bearer и cookie credentials

Свойство	Bearer JWT	HttpOnly cookie
Кто добавляет credential	JavaScript через Authorization	Browser автоматически
Доступ JavaScript к credential	Да, если token находится в JS state	Нет при HttpOnly
credentials: include	Не нужен для Authorization header	Нужен для cross-origin cookie fetch
CSRF	Обычно ниже при ручном	Требуется отдельного внимания

Свойство	Bearer JWT	HttpOnly cookie
	Bearer header	

18. Изучите CSRF-сценарий

Cookie может автоматически прикладываться browser к подходящему запросу. Поэтому state-changing endpoints при cookie-based Authentication требуют CSRF-защиты. Не выполняйте создание, изменение или удаление данных через GET.

- Используйте POST, PUT, PATCH или DELETE для изменения состояния.
- Используйте SameSite как дополнительный защитный слой.
- Для подходящих архитектур применяйте CSRF token.
- Проверяйте Origin или Referer для чувствительных state-changing requests.
- Используйте HTTPS и Secure cookies в production.

19. Самостоятельное задание

27. Объясните, почему ports 5500 и 3000 создают разные origins.
28. Покажите CORS error при отключённом middleware.
29. Разрешите только http://localhost:5500.
30. Найдите Access-Control-Allow-Origin в DevTools.
31. Найдите OPTIONS preflight для JSON/Bearer request.
32. Определите Access-Control-Request-Method.
33. Определите Access-Control-Request-Headers.
34. Докажите, что origin http://localhost:5600 не разрешён.
35. Проверьте credentialed request с credentials: include.
36. Найдите HttpOnly demo cookie.
37. Объясните различие CORS, Authentication и Authorization.
38. Объясните, почему cookie-based Authentication требует CSRF-защиты.

Требования к результату

- Frontend и backend работают на разных origins.
- Установлен и настроен cors.
- Разрешён только заданный frontend origin.
- JSON и Authorization headers разрешены явно.
- Студент находит OPTIONS preflight.
- Студент читает Access-Control-* headers.
- Другой origin не получает разрешение CORS.
- Credentialed request использует credentials: include.
- Credentialed CORS использует explicit origin.
- Demo cookie имеет HttpOnly.
- Студент понимает роль SameSite и Secure.
- Студент объясняет CSRF-риск cookie-based Authentication.

Критерии оценивания

№	Критерий
1	Студент корректно определяет origin
2	Показан CORS error без настройки

№	Критерий
3	Пакет cors установлен
4	Разрешён только frontend origin
5	methods настроены
6	Content-Type и Authorization разрешены
7	GET cross-origin работает
8	Найден OPTIONS preflight
9	Студент объясняет назначение preflight
10	Другой origin блокируется browser CORS policy
11	credentials: include используется в cookie experiment
12	Access-Control-Allow-Credentials присутствует
13	HttpOnly demo cookie недоступна через document.cookie
14	Студент различает CORS и server-side access control
15	Студент объясняет CSRF-риск и базовые меры защиты